Parallel & Distributed Computing Lab

IMSciences

Lab Instructor: Saim Sanan

Week 5, Lab 2

Architectures and a Unix pipe

---

1 Ideas

Last lab coded client–server sockets on `localhost`. This lab reviews the three architectures with examples, then uses a pipe: message passing with no IP and no port.

- **Client–server** — one server waits; clients dial (web, email apps, Lab 1 sockets).
- **Peer-to-peer (P2P)** — equal peers swap data. In BitTorrent the file is handled as pieces; a peer is a machine in the swarm; a seeder has the complete file and uploads pieces to others. Uploads go peer to peer, not to one central movie server.
- **Hybrid** — a server helps with login or finding peers; heavy traffic still moves among peers (BitTorrent with a tracker).

A pipe is a one-way tube. `pipe(fd)` creates it: `fd[1]` is the write end, `fd[0]` is the read end. `fork` copies the process into a parent (`pid > 0`) and a child. In this program the parent writes and the child reads. Real-world twin: `ls | grep`.

2 Commands

```
nano ipc_pipe.c
gcc ipc_pipe.c -o ipc
./ipc
```

3 Pipe — `ipc_pipe.c`

```
#include <stdio.h>
#include <unistd.h>
#include <string.h>

int main(void) {
    int fd[2];
    pid_t pid;
    char message[] = "Hello from parent";
    char buffer[100];

    pipe(fd);
    pid = fork();

    if (pid > 0) {
        close(fd[0]);
        write(fd[1], message, strlen(message) + 1);
        close(fd[1]);
    } else {
        close(fd[1]);
        read(fd[0], buffer, sizeof(buffer));
```

```
        printf("Child received: %s\n", buffer);
        close(fd[0]);
    }

    return 0;
}
```

Expected output:

```
Child received: Hello from parent
```

4 Check

1. Name the three architectures. Give one example each. Which architecture did Lab 1 code?
2. In BitTorrent, state what a piece, a peer, and a seeder are, and where uploads go.
3. In the pipe program, state which end is write, which end is read, and which process writes.